

Rapid Revision Sheet

Event Loop Phases

- * Timers: `setTimeout()`, `setInterval()`
- * Pending Callbacks: system-level callbacks, errors
- * Idle/Prepare: internal use
- * Poll: I/O callbacks, waits for new I/O events
- * Check: `setImmediate()` callbacks
- * Close Callbacks: socket close, cleanup events

Key Formulas and Definitions

- * Event Loop Cycle: Timers ? Pending Callbacks ? Idle/Prepare ? Poll ? Check ? Close Callbacks
- * Poll Phase: executes I/O callbacks, waits for new I/O events

Memory Trick

- * Timers: alarm clocks ?
- * Poll: waiting for delivery ?
- * Check: "run immediates now!" ?

Call Stack vs Callback Queue vs Microtask Queue

- * Call Stack: execution engine, LIFO
- * Callback Queue: stores `setTimeout`, `setInterval`, I/O callbacks
- * Microtask Queue: stores `Promise.then()`, `Promise.catch()`, `queueMicrotask()`

Priority Order

1. Call Stack
2. `process.nextTick()` (Node.js)
3. Microtask Queue (Promises)
4. Callback Queue (Timers, I/O)

Golden Rules

- * If stack is not empty, nothing runs

- * After stack empty, run all microtasks
- * Then, take one callback from callback queue

Real-World Analogy

- * Call Stack: chef cooking ?
- * Microtasks: urgent VIP orders ?
- * Callback Queue: normal orders ?